Relazione Progetto Sistemi Intelligenti per Internet

Questo progetto tratta un sistema software con interfaccia Web, in particolare un Decision Support System, che permette ad un oncologo di caricare delle analisi cliniche di un paziente in cura, e lo aiuta a capire se il paziente è oncologico oppure no, e con che probababilità. Tale progetto, si basa sul linguaggio di programmazione Python, poiché è il linguaggio più utilizzato nell'ambito del Machine Learning, che è ciò su cui si basa la maggior parte progetto. Inoltre, implementa anche la XIA, che spiega il motivo della decisione presa dal sistema.

Architettura software utilizzata

Tale applicazione è strutturata secondo il paradigma Object-Oriented e l'architettura Client-Server, garantendo una rigorosa Separation of Concerns (SoC) attraverso il pattern Model-View-Controller (MVC).
Il Model si occupa della persistenza dei dati e della logica di dominio: è implementato tramite il DBMS open-source PostgreSQL (molto usato in ambito medico) e usa il modello di Machine/Deep Learning per l'inferenza dei dati.
La View è responsabile della presentazione e dell'interazione con l'utente: in particolare è sviluppata tramite il framework Streamlit, che rende del tutto invisibile agli sviluppatori la complessità di HTML, CSS e JavaScript, fornendo un'interfaccia reattiva ed soprattutto intuitiva. Ciò viene arricchito dalla visualizzazione di grafici per la eXplainable AI (XAI) e referti in PDF: in questo modo l'utente può capire i motivi che hanno portato alla decisione del modello: infatti, questo passa da essere un sistema di Intelligenza Artificiale puro ad un vero e proprio Sistema di Supporto alle Decisioni.
Il Controller, che è stato sviluppato in Python 3.12, è il direttore di orchestra del sistema vero e proprio: difatti gestisce le richieste da parte del frontend, interroga il database, elabora i dati tramite l'IA e invoca anche dei processi esterni per generare il referto in PDF, sfruttando un compilatore LaTeX, ottimo per la generazione di documenti PDF.
L'intera architettura è containerizzata tramite Docker, pratica molto comune in ambiente di sviluppo, poiché assicura la massima portabilità, isolamento e scalabilità dell'ambiente di esecuzione.

Entità e relazioni

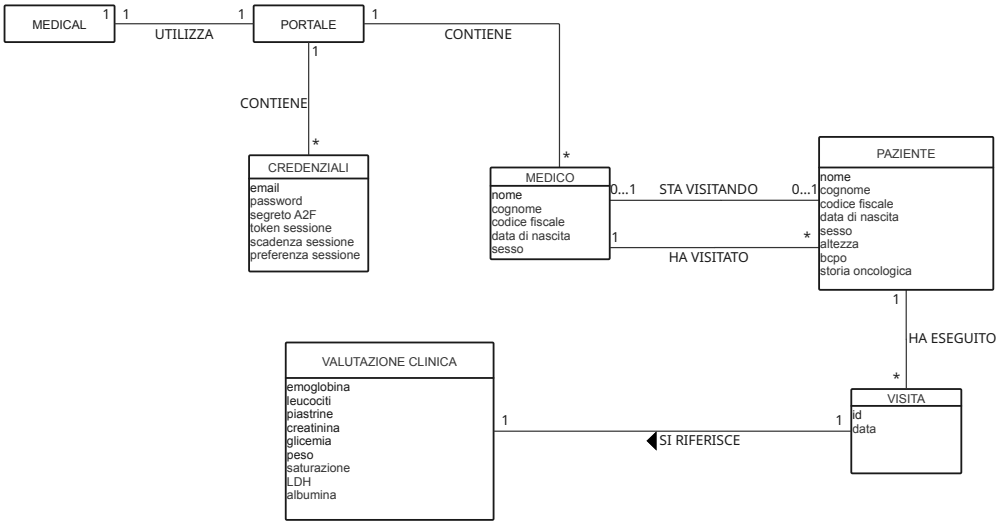


Figura 1: Modello di dominio

Il sistema deve poter memorizzare ogni medico, con delle credenziali, a cui viene associata una email. Ogni medico contiene un nome, un cognome, una data di nascita, un codice fiscale ed il sesso. Le credenziali invece contengo, l'email, la password criptata, dei token per l'autentificazione a due fattori e per la sessione, per evitare logout al reload della pagina e gestire la sessione per il sito.
Il medico può selezionare ogni paziente che ha in cura, che inserisce il suo nome, cognome, codice fiscale, data di

nascita, sesso, altezza, se è un soggetto BCPO (BroncoPneumopatia Cronica Ostruttiva) e se in passato ha avuto dei precedenti oncologici. Il paziente è in cura presso un solo medico.
Il medico esegue delle visite al paziente, in cui viene memorizzata la data ed un identificativo per essere memorizzata. A questo punto, ad ogni visita gli viene associata, una propria valutazione clinica, in cui vengono memorizzati questi valori non obbligatori eccetto per il primo: peso, emoglobina, leucociti, piastrine, creatinina, glicemia, saturazione, LDH ed albumina.

Caso d'uso "processa analisi"

Il sistema possiede diversi casi d'uso, come l'inserimento del nuovo paziente, di una nuova visita, la registrazione di un nuovo medico. Tuttavia, quello soggetto di studio è quello della predizione.
Caso d'uso: Processa Analisi

1. Un medico vuole eseguire la predizione delle analisi effettuate ad un paziente.
2. Il medico esegue il login, con l'A2F ed il Sistema mostra i suoi dati e i pazienti in cura.
3. Il medico seleziona il paziente a cui vuole effettuare la predizione: il Sistema mostra la cartella clinica e l'elenco delle visite effettuate.
4. Il medico seleziona la predizione delle analisi, il Sistema carica le analisi e avvia il modello di Machine Learning, che invia al Sistema il risultato della predizione.
5. Il medico può scaricare il referto in formato PDF.

Di seguito, viene mostrato il seguente diagramma delle classi, dove GestoreDB permette la connessione con il DBMS.

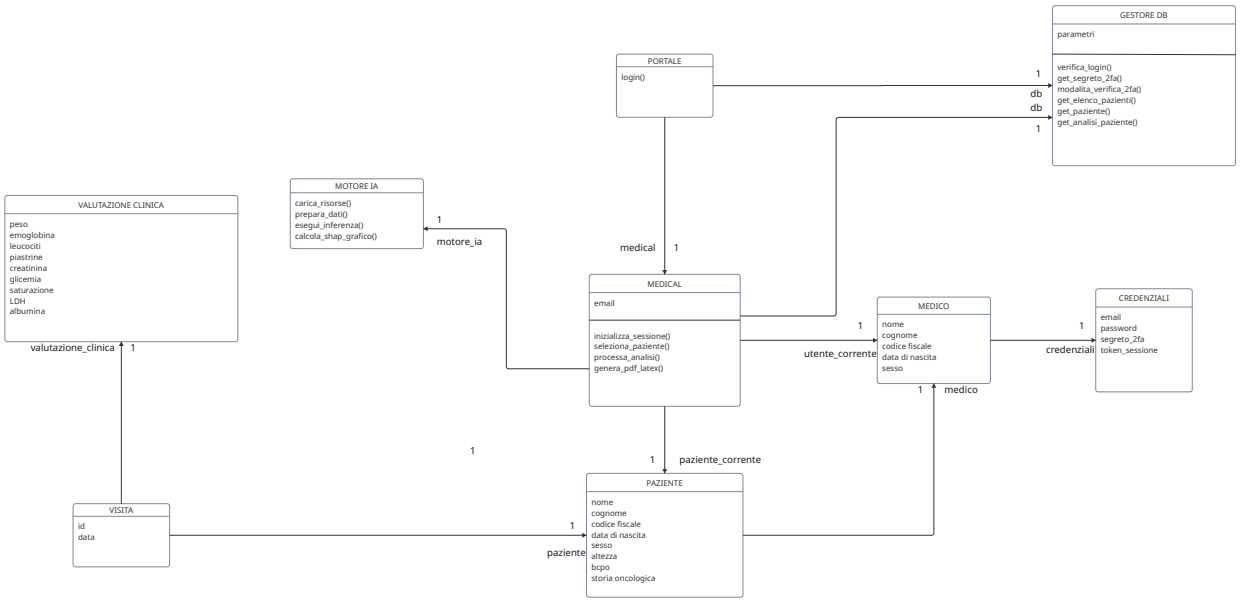
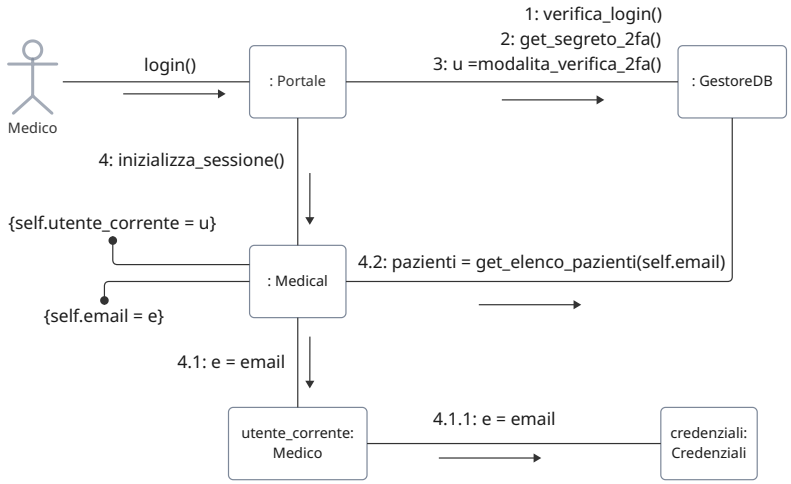


Figura 2: Diagramma delle classi

Ora, verrà trattato nel dettaglio ogni punto del caso d'uso, illustrando anche il relativo diagramma di sequenza di sistema.
Per prima cosa, il medico registrato deve effettuare il login. Il sistema usa come prima validazione per le email e per le password le espressioni regolari, per evitare di interrogare il database nel caso sia le email (*stringa@stringa.stringa*) che per il numero di caratteri della password. Durante la prima registrazione, il medico tramite QRCode configura la verifica a due fattori, usando l'app Google Authenticator, per garantire maggiore sicurezza, dato che tratta comunque dati sensibili. Per gli altri accessi, invece, viene richiesto il codice dall'applicazione. Una volta effettuato il login, verranno stampati tutti i pazienti in cura del medico.



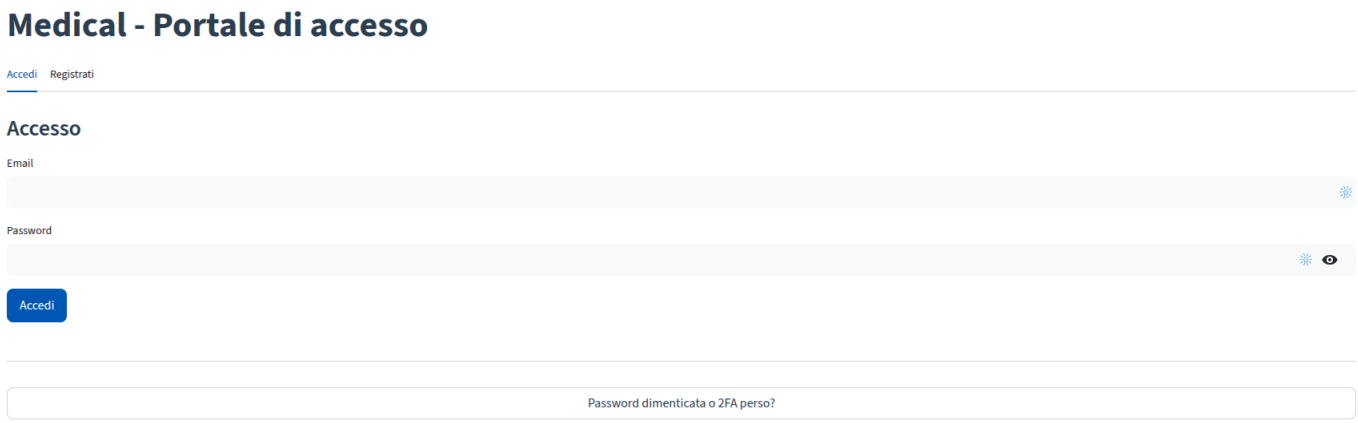


Figura 3: Diagramma di sequenza di sistema e pagina Web del login

A questo punto, il medico può selezionare il paziente su cui deve elaborare il modello di Machine Learning e vengono stampate le visite effettuate di quel paziente in ordine decrescente per data di visita. Il paziente viene dunque selezionato in base al suo codice fiscale, poiché è univoco per ogni paziente. Una volta fatto ciò, vengono stampati i valori delle ultime visite e la sua cartella clinica.

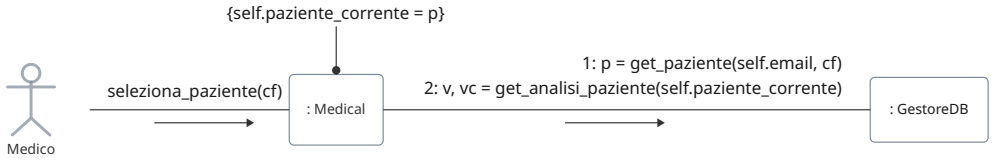


Figura 4: Diagramma di sequenza di sistema della selezione del paziente

Quando il medico ha inserito tutto nel sistema, può avviare il modello di Machine Learning. Vengono passati tutte le analisi ed alcuni valori clinici, con delle conversioni. Viene poi stampato il risultato dell'esito e la probabilità di positività del paziente.

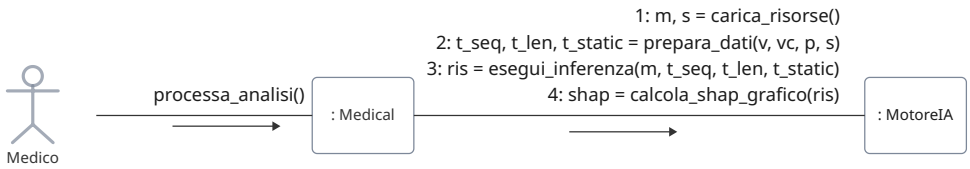


Figura 5: Diagramma di sequenza di sistema dell'avvio del modello di Machine Learning

In particolare:

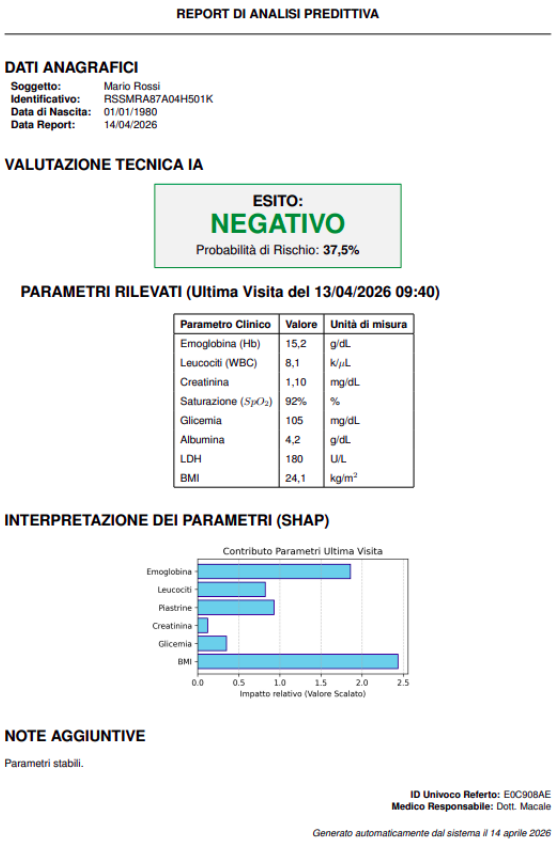
- se la probabilità è minore od uguale del 40 %, allora il paziente viene classificato come negativo;
- se la probabilità è compresa tra il 40 % (escluso) ed il 70 % (incluso), allora viene classificato come a rischio;
- se la probabilità è maggiore del 70 %, allora il paziente viene classificato come positivo.

Tuttavia, tutto ciò verrà trattato nel dettaglio in un paragrafo dedicato.

Figura 6: Tipologie di risultati ottenuti

Infine, il medico può scaricare il referto in formato PDF.

Figura 7: Diagramma di sequenza di sistema del download di un referto



REPORT DI ANALISI PREDITTIVA

DATI ANAGRAFICI

Soggetto: Mario Rossi
Identificativo: RSSMRA87A04H501K
Data di Nascita: 01/01/1980
Data Report: 14/04/2026

VALUTAZIONE TECNICA IA

ESITO:
NEGATIVO
Probabilità di Rischio: **37,5%**

PARAMETRI RILEVATI (Ultima Visita del 13/04/2026 09:40)

Parametro Clinico	Valore	Unità di misura
Emoglobina (Hb)	15,2	g/dL
Leucociti (WBC)	8,1	k/μL
Creatinina	1,10	mg/dL
Saturazione (SpO_2)	92%	%
Glicemia	105	mg/dL
Albumina	4,2	g/dL
LDH	180	U/L
BMI	24,1	kg/m^2

INTERPRETAZIONE DEI PARAMETRI (SHAP)

NOTE AGGIUNTIVE

Parametri stabili.

ID Univoco Referto: E0C908AE
Medico Responsabile: Dott. Macale

Generato automaticamente dal sistema il 14 aprile 2026

Figura 8: Esempio di un referto generato da un paziente placeholder

Modello di Machine Learning

Adesso, si entra nel cuore del progetto: il modello di Machine Learning. Prima di parlare del modello vero e proprio, occorre capire bene come è strutturato il dataset a cui si fa riferimento, necessario per capire le motivazioni della scelta del modello.

Dataset di riferimento

Il dataset è MIMIC-IV da PhysioNet, che raccoglie una lista di pazienti di un ospedale statunitense, divedendo i pazienti in reparto ospedaliero ed i pazienti in reparto in terapia intensiva. Le difficoltà iniziali sono:

- l'intero dataset è composto da diversi file .csv con tabelle anche di più 40 GB;
- eseguire la join tra le varie tabelle è molto complesso, dato che alcune tabelle non contengono chiavi primarie.

Per questo motivo, sono state caricate le tabelle più rilevanti su PostgreSQL, per poi esportare il csv completo, necessario per allenare il modello. Le tabelle utilizzate (in ordine alfabetico in inglese) sono:

- *admissions*, che riporta le ammissioni dei pazienti in pronto soccorso;
- *diagnoses_icd*, che riporta i codici icd delle diagnosi;
- *labevents*, che riporta gli esami di laboratorio del paziente;
- *omr*, che riporta i dati clinici del paziente (es. peso, altezza, pressione sanguigna);
- *piatients*, che riporta i dati anagrafici del paziente;
- *procedures_icd*, che riporta le informazioni riguardanti ad esempio precedenti oncologici.

Analisi esplorativa dei dati

La prima fase quando si esegue l'analisi esplorativa dei dati, più comunemente chiamata EDA (Explorative Data Analysis), è la gestione dei valori nulli. Tuttavia, in questo caso i valori nulli sono fondamentali da tenere. Per questo motivo, viene sostituito con 0 e viene inserita una maschera booleana, per dire se il dato è mancante (indicato con 0), oppure no (indicato con 1).
Successivamente, viene calcolata la correlazione delle feature, con la matrice di correlazione ed il VIF.

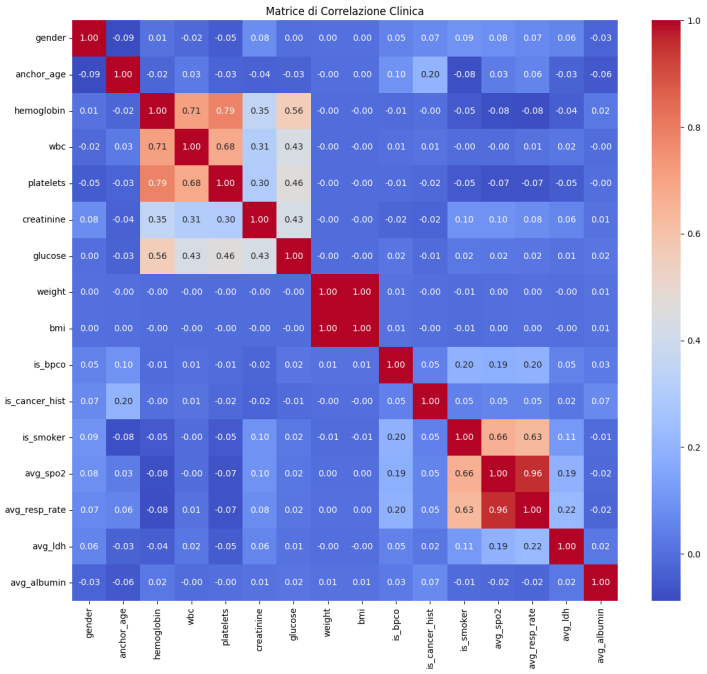


Figura 9: Matrice di correlazione

Feature	VIF
weight	270,275407
bmi	270,271329
avg_spo2	12,547373
avg_resp_rate	11,961291
hemoglobin	3,454344
platelets	2,891586
wbc	2,243069
is_smoker	1,847033
glucose	1,603366
creatinine	1,298581
anchor_age	1,105792
is_bpco	1,069602
avg_ldh	1,069346
is_cancer_hist	1,062703
gender	1,047048
avg_albumin	1,017267

Tabella 1: Calcolo del VIF

Come si può notare, ci sono delle feature che hanno come VIF 270 o 12: con VIF maggiore od uguale a 10 si ha multicolinearità grave, perciò sono state rimosse *weight* e *avg_resp_rate*: entrambe le feature si possono ricavare da altre, quindi generano solo rumore. Non a caso BMI (in italiano) si calcola dal peso, perciò ci si aspetta che abbiano un VIF altro se sono presenti entrambi. Rimuovendo queste feature, non si ha una multicolinearità rilevante.

Feature	**VIF**
hemoglobin	3,449920
platelets	2,891582
wbc	2,235239
avg_spo2	1,878255
is_smoker	1,846963
glucose	1,603135
creatinine	1,295873
anchor_age	1,099846
is_bpco	1,066946
avg_ldh	1,069346
is_cancer_hist	1,062510
gender	1,044424
avg_albumin	1,016640
bmi	1,000222

Tabella 2: Calcolo del VIF dopo la rimozione delle feature con multicolinearità

In seguito, avviene la gestione degli outliers: molto dannosi nella predizione, perché il modello non sa come trattare i casi rari, generando dei falsi positivi o dei falsi negativi. Per fare ciò, si esegue un box-plot, in base al target, che per semplicità viene rappresentato come sano o malato. Inoltre, si eliminano gli eventuali errori di battitura (es. un peso di 500 kg)

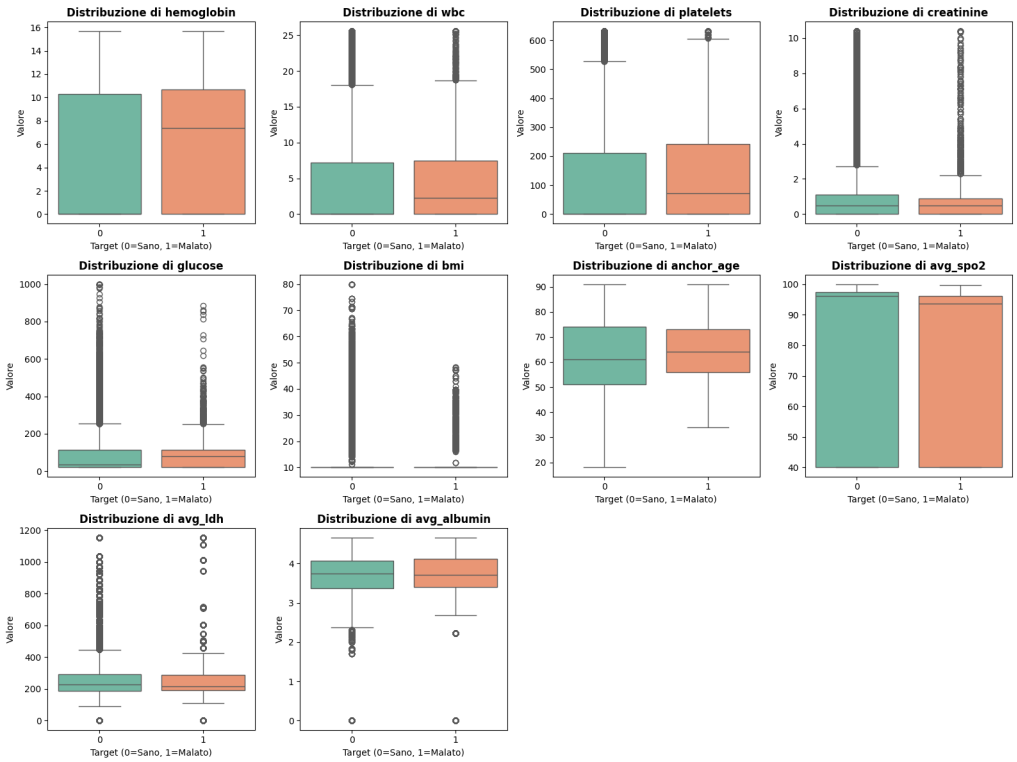


Figura 10: Box-plot per individuare gli outliers

Come si può notare, sono presenti molti outliers: in ambito medico è del tutto normale, poiché molto spesso si analizzano dei valori che sono fuori range: addirittura da lì che si identifica un malato non necessariamente oncologico. Tuttavia, è necessario rimuoverli, altrimenti il modello non saprà gestirli: per fare ciò si esegue una standardizzazione, in modo tale da avere una distribuzione con valor medio nullo e varianza unitaria.

Infine, si esegue la selezione delle feature, molto importante per capire quali sono effettivamente gli attributi rilevanti nella predizione: le feature inutili, producono solamente rumore. Per fare ciò, si allena un classificatore RandomForest, che valuta l'importanza di ogni feature con l'indice di Gini. I risultati usciti sono riportati nel barplot che segue.

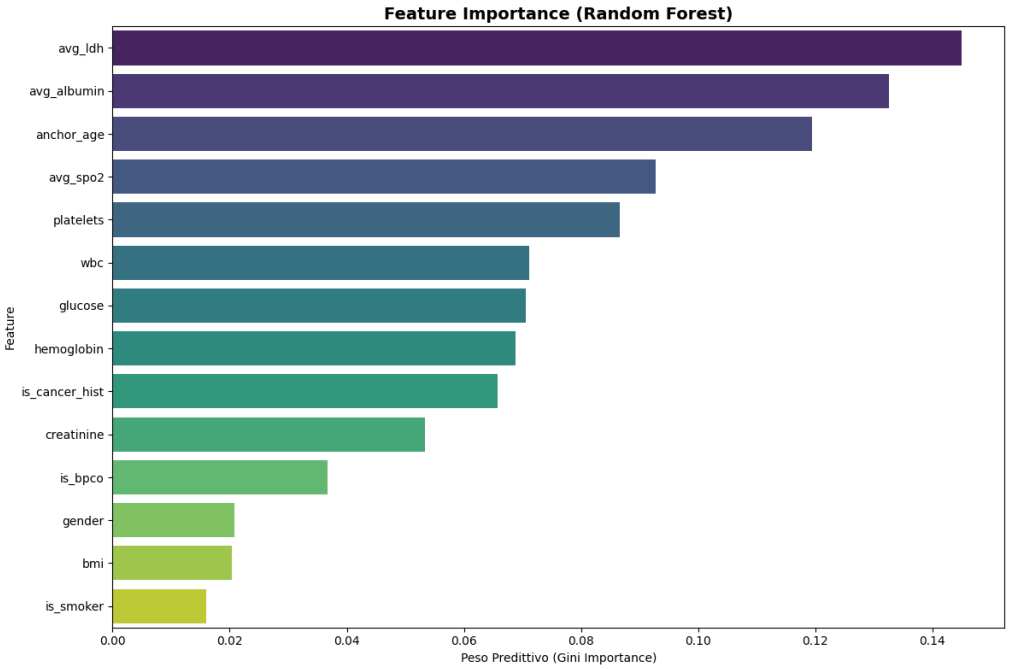


Figura 11: Importanza di ogni feature calcolata con l'indice di Gini su un classificatore RandomForest

Come si evince dal grafico, le feature *is_smoker*, *bmi* e *gender* risultano: meno determinanti per il classificatore. Tuttavia le feature che sono state escluse sono solamente *is_smoker* e *gender*, poiché *bmi* è comunque importante in ambito oncologico per la valutazione dello stato generale del paziente.

Scelta del modello, fase di traning e risultati ottenuti

Per quanto riguarda la scelta del modello, è necessario comprendere nel dettaglio il problema di riferimento. In questo contesto, si ha un problema di classificazione binaria, in cui si deve predire se un paziente è negativo oppure positivo, a partire dalla sua storia clinica. In particolare, il dataset su cui è stato allenato il modello si compone appuntu della storia clinica su dei pazienti di un ospedale statunitense: perciò sono registrate ogni misurazione ed ogni analisi registrate sul paziente. La variabile target viene individuata se almeno nella diagnosi risulta avere un tumore visibile dall'RX al torace: sicuramente ciò restringe il campo di ricerca e quindi il modello risulta sicuramente più capace di individuare poche tipologie di cancro. L'ospedale di riferimento, però, possiede tantissimi reparti e non solo quelli di oncologia, perciò già solamente dividere i pazienti con qualsiasi tipologia di tumore racchiuderebbe pochi casi. Facendo questa ulteriore divisione, si ottengono pochissimi pazienti positivi. Ciò viene riportato della tabella che segue.

Tipologia di paziente	Numero	Percentuale
Negativo	4654	97 %
Positivo	140	3 %

Tabella 3: Analisi del target

Questo è un problema serio per le performance del modello e complica sicuramente la scelta del modello migliore. Analizzando i dati, si nota che sono presenti delle feature che sono time-sensitive: perciò la scelta più coerente e performante è una RNN (Recurrent Neural Network), che appertiene al Deep Learning. In particolare è stata scelta una LSTM (Long Short-Term Memory), poiché è necessario elaborare delle vere e proprie sequenze temporali. Dalla LSTM, viene concetenata una rete neurale fully-connected MLP (Machine Learning), che invece esegue la classificazione vera e propria.
A questo sono state divise le feature in tre categorie principali:

- feature temporali, che solo le feature che sono time-sensitive, che comprendono i parametri fisiologici e di laboratorio monitorati nel tempo per catturare le dinamiche evolutive del paziente, che costituiscono l'input principale elaborato dalla componente LSTM della rete;
- maschere, che fungono da indicatore di presenza, che indica se il valore delle feature è presente oppure è mancante, fondamentale per la natura irregolare e frammentata dei dati clinici;
- feature cliniche, che indica il contesto storico statico del paziente, che fungono da baseline per il classificatore.

Per via del problema dello sbilanciamento estremo del dataset, effettuare una suddivisione netta tra training set, e validation set risulta quasi impossibile. Per questo motivo, è stato scelto di optare per la K-fold cross-validation (K = 5): tuttavia viene preso un 15 % di test set, indispensabile per effettuare l'analisi dei risultati. In particolare, la K-fold cross validation consiste nella suddivisione del dataset in K sottoinsiemi, in cui il training set è composto da $K-1$ sottoinsiemi e la parte rimanente è il validation set. Perciò avvengono K addestramenti di 25 epoche ciascuno con early stopping con pazienza pari a 3, per evitare che il sistema vada in overfitting, imparando a memeoria i dati senza comprendere il vero funzionamento. Per ogni addestramento, vengono calcolate le metriche di train Loss e validation loss, le più importanti per comprendere se il sistema sta andando in overfitting. La precision, la recall e l'accuracy, invece vengono calcolate in seguito alla fine, poiché essendo il dataset così sbilanciato, perdono di significatività. A questo punto si esegue il cosidetto OOF (Out-Of-Fold), che somma le performance reali del sistema. Di seguito, ne viene mostrato l'andamento del train e validation loss.

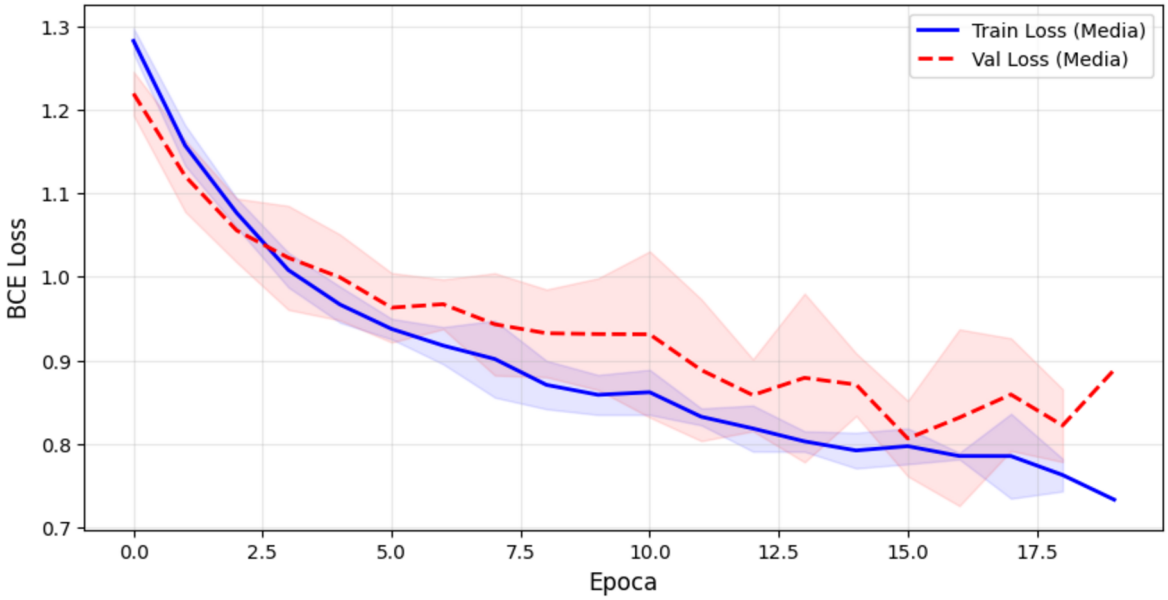


Figura 12: Andamento del train loss e del validation loss

Come si può notare, il grafico mostra una curva di apprendimento estremamente regolare: la train Loss decresce in modo costante, indicando che il modello sta imparando efficacemente i pattern presenti nei dati clinici. In parallelo, la validation loss segue lo stesso trend discendente senza mostrare divergenze brusche. Il modello non sta soffrendo né di overfitting né di underfitting, poiché le curve siano vicine tra loro. Inoltre, le aeree di varianza attorno alle medie dimostra che il classificatore non dipende da uno split di dati ben specifico, bensì reagisce in modo uniforme a diverse porzioni del dataset. Infine, si nota che grazie all'early stopping, il training è stato interrotto prima, poiché non ci sono stati miglioramenti per 3 epoche consecutive.
Ora si rende necessario stabilire la soglia di decisione: di default è impostata a 0,5, ma nella situazione attuale di uno sbilanciamento tale, si rende necessario alzarla a 0,55, altrimenti generebbe pochissimi falsi negativi, il che è ottimo, ma ci sarebbero più falsi positivi che negativi. In altre parole, tantissimi pazienti sani verrebbero classificati come oncologici.
Per prima cosa, si rappresenta la AUCROC (Area Under Curve Receiver Operating Characteristic), che mette in relazione la sensibilità True Positive Rate e l'impatto dei falsi allarmi (False Positive Rate) al variare della soglia. Il valore ottenuto indica la probabababilità che il modello più alto ad un paziente effettivamente positivo rispetto ad un paziente sano.
Di seguito, viene mostrata una rappresentazione grafica ed il risultato ottenuto dal grafico stesso, con i relativi commenti e valutazioni.

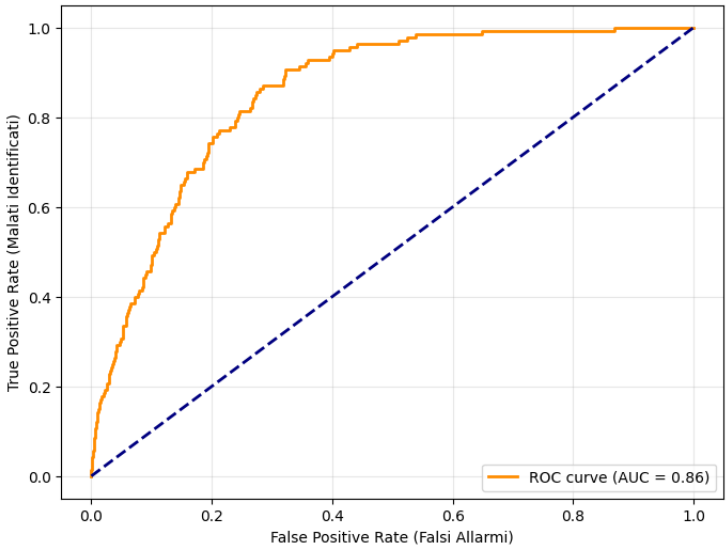


Figura 13: Grafico dell'AUCROC

$$\text{AUCROC} = 0{,}8562 \tag{1}$$

Infine, ne viene calcolata la matrice di confusione, che mostra ogni paziente come è stato classificato e come doveva essere classificato: ovviamente l'obiettivo è massimizzare il numero di pazienti sulla diagonale principale, perché rappresenta rispettivamente i veri negativi ed i veri positivi. Ne viene, inoltre, riportato il calcolo dell'Accuracy e della Balanced Accuracy.

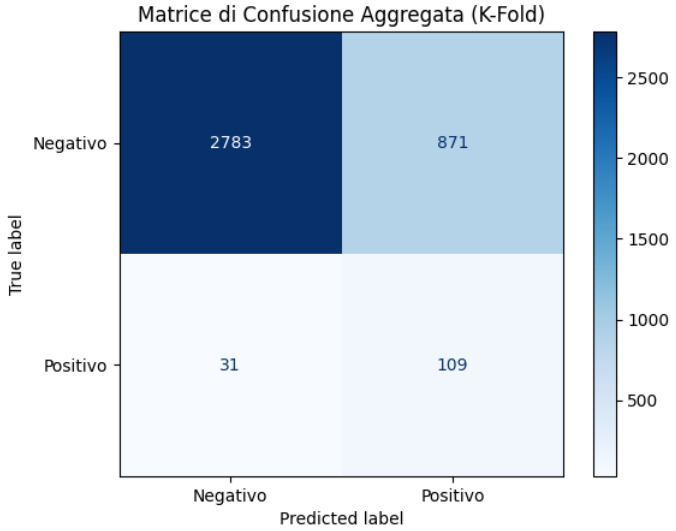


Figura 14: Matrice di confusione

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} = \frac{109 + 2783}{109 + 2783 + 871 + 31} \approx 0{,}76 \tag{2}$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} = \frac{109}{109 + 31} \approx 0{,}78 \tag{3}$$

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}} = \frac{2783}{2783 + 871} \approx 0{,}76 \tag{4}$$

$$\text{Balanced Accuracy} = \frac{\text{Recall} + \text{Specificity}}{2} = \frac{0{,}78 + 0{,}76}{2} \approx 0{,}77 \tag{5}$$

Dalle metrica appena riportate, avere una Balanced Accuracy pari al 77 % dimostra che, nonostante il forte sbilanciamento del dataset, il modello è molto più efficace di un classificatore casuale.

Interpretazione del modello e Explainable AI

Infine, per evitare di avere un sistema a black-box, non molto apprezzato in ambito medico, poiché si vuole comprendere le motivazioni della scelta della predizione del modello, viene implementata la cosiddetta Explainable AI, abbreviata in XIA: quella implementata è la SHAP (Shapely Additive Explanations), che permette di quantificare il contributo di ogni singola feature alla predizione finale, basandosi sulla teoria dei giochi. Di seguito, vengono riportati due grafici, che riportano rispettivamente l'analisi globale e la contribuzione locale.

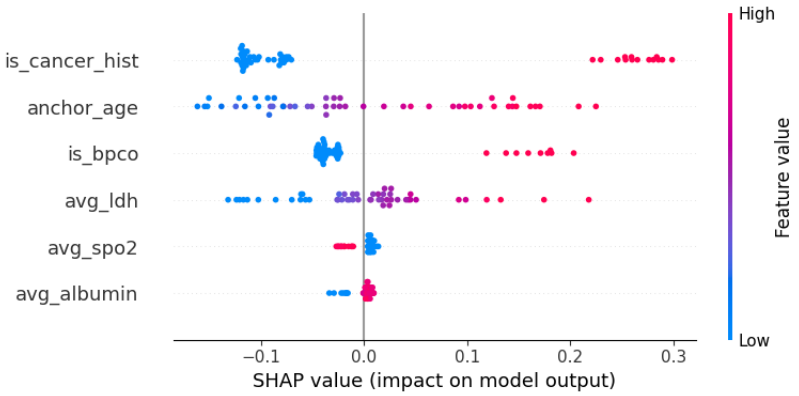


Figura 15: Analisi globale dell'importanza delle feature

Per prima cosa, si parte dell'analisi globale:

- *is_cancer_hist*, che è intuitivamente la variabile più influente, infatti graficamente aumenta la probababilità di outcome (punti rossi a destra), mentre l'assenza (punti blu a sinistra) la riduce;
- *anchor_age*, infatti all'aumentare dell'età aumenta il rischio;
- *is_bpco*, difatti la presenza di BCPO è un forte indicatore positivo.

Infine, per quanto riguarda l'analisi locale, mostra come ogni paziente ha influenzato la decisione del modello.

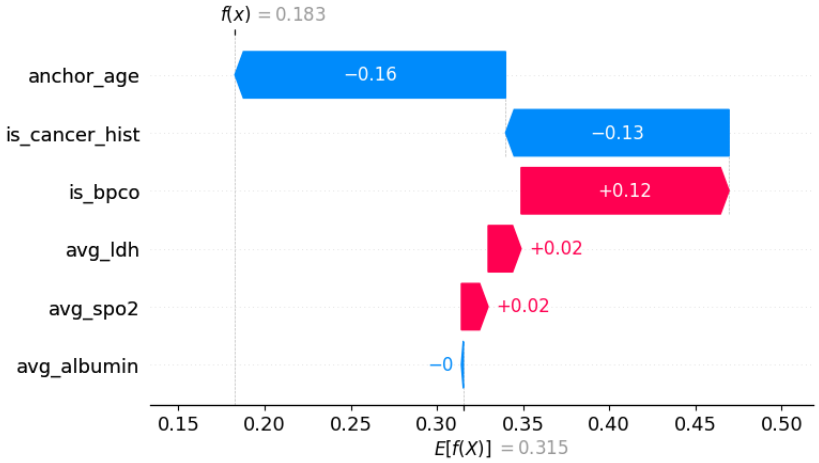


Figura 16: Analisi locale di un singolo paziente

In particolare si fa riferimento:

- il valore base ($E\,[f(X)] = 0{,}315$) rappresenta la probababilità media predetta del modello del dataset;
- il valore finale ($f(x) = 0{,}183$) è la probabilità specifica calcolata per questo paziente.

I contributi che aumentano maggiormente il rischio sono *is_bpco*, *avg_ldh* e *avg_spo2*, mentre i contributi che lo hanno diminuito sono *anchor_age* e *is_cancer_hist*.